

UNIT-1

No.

Date

Introduction - What is an Operating System

Simple Batch Systems

Multiprogrammed Batch Systems

Time Sharing Systems

Personal Computer Systems

Parallel Systems

Distributed Systems

Realtime Systems

O.S - A Resource Manager.

Memory Organization & Management

Memory Organization

Memory Hierarchy

Memory Management Strategies

Contiguous vs Non Contiguous Memory allocation

Partition Management Techniques

Logical vs Physical Address Space

Swapping

Paging

Segmentation

Segmentation with Paging.

Virtual Memory - Demand Paging

Page Replacement

Page Replacement Algorithms

Performance of Demand Paging

Thrashing

Demand Segmentation

Overlay Concepts.

1. An operating system acts as an intermediate between the user and computer hardware. The purpose of an operating system is to provide an environment in which a user can execute programs in a convenient or efficient manner.

The operating system is a software that manages the computer hardware.

DEFINITION: OS is a program that controls execution of application programs and acts as an interface between user and hardware.

An OS is concerned with the allocation of resources and services, such as memory, processors, devices, etc.

The OS includes programs to manage these resources such as traffic controller, a scheduler, memory manager, etc.

Evolution of Operating Systems

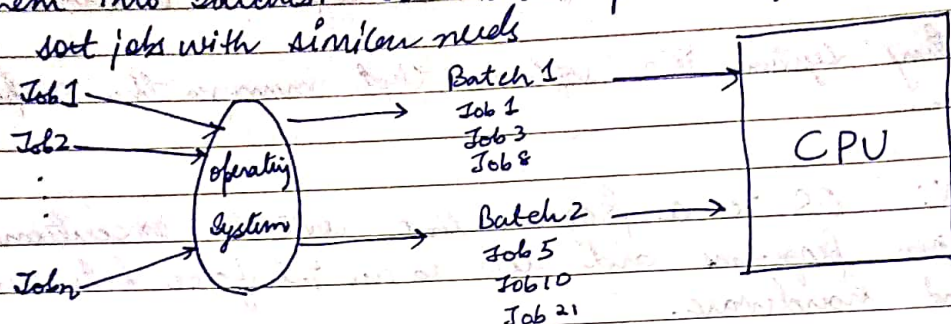
	Device Used	Type of OS
First Generation	Vacuum Tubes	Plug Boards
Second Generation	Transistors	Batch Systems
Third Generation	ICs	Multi programming OS
Fourth Generation	Large Scale Integration	PC

TYPES OF OPERATING SYSTEMS

- ① Batch Operating Systems
- ② Time-Sharing Operating Systems
- ③ Distributed Operating Systems
- ④ Network Operating Systems
- ⑤ Real time Operating Systems.

① Batch Operating System

There is an operator that takes similar jobs and groups them into batches. It is the responsibility of the operator to sort jobs with similar needs.



Advantages of Batch Systems -

- no manual intervention is needed for batch execution
- Jobs run one after another saving time for activities like loading of computer.

Disadvantages

- Lack of Interaction b/w User and Job
- No concept of priority
- Hard to debug.

② Multiprogrammed Operating Systems

Multiprogramming is an extension to batch system where CPU is always kept busy.

In a multiprogramming environment for the time that a process does IO, the CPU can start executing another process, it doesn't need to remain idle.

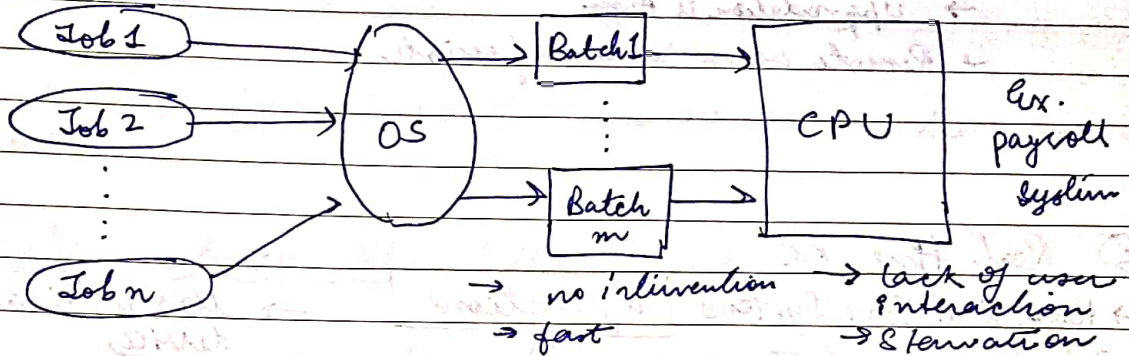
This improves efficiency of the system.

③ Time Sharing Operating Systems -

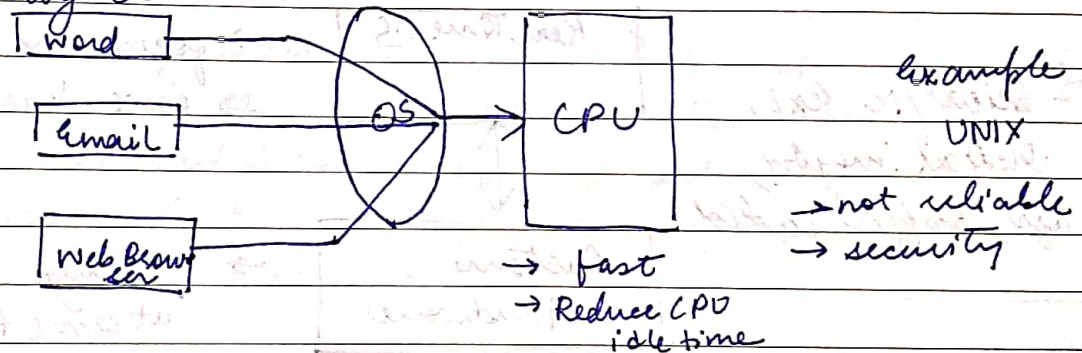
Each task is given some time to execute, so that all tasks work smoothly. Each user gets time of the CPU as if they use single system.

The time that is given to the task is called quantum. After this time gets over OS switches to another task.

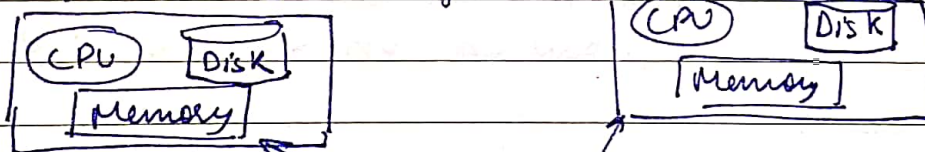
① Batch Operating Systems



② Time Sharing OS



③ Distributed OS / loosely coupled systems



example: LOCUS

→ language not well defined
→ expensive

→ all systems independent

→ resource sharing

Neelgagan no delay.

① Network OS

Client 1

Client 2

Client 3

Client 4

File
System
Server

ex

Mac OS X

Linux

Windows Server
2003

- centralised server is stable
- security concerns are handled
- Upgradation is easy
- Remote access to server possible

- Costly
- Server dependence
- Regular maintenance

③ Real Time OS

- Hard Real Time Systems
- Soft Real Time Systems

eg - Scientific exp,
Medical imaging,
air traffic control

Applications

Real Time OS

Custom
Hardware

- Max utilization of devices
- less time for shifing
- focus on application
- error free

- not many tasks run at same time
- complex algorithms

Functions of Operating Systems

No.

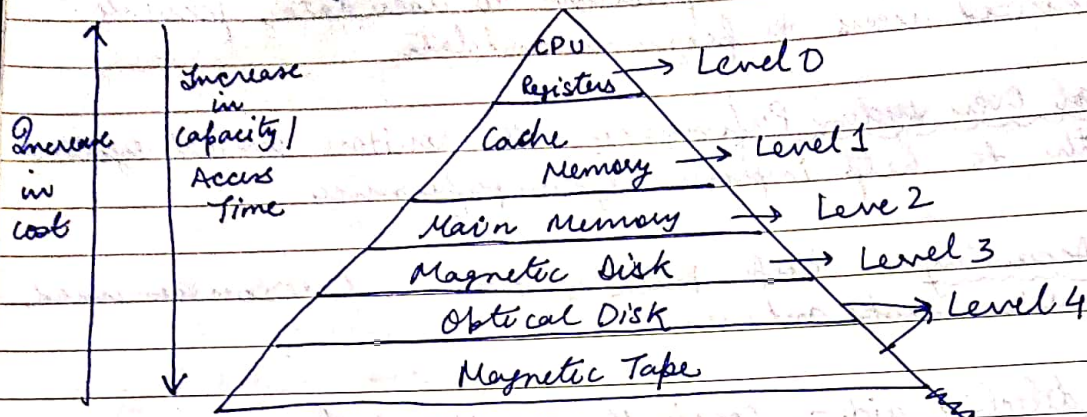
Date

- ① Security - Uses password protection to user data, prevents unauthorized access to programs and data.
- ② Control Over system Performance - It monitors overall system health to help improve system performance.
- ③ Job Accounting - Keeps track of time and resources used by various tasks and users.
- ④ Error detecting aide - Constantly monitors system to detect errors.
- ⑤ Coordination between other Software and Users
- ⑥ Memory Management - OS manages the main memory or the primary memory.
It keeps track of main memory, i.e. which bytes are used by which programs, the memory addresses that have already been allocated and those that are not yet used. OS decides the order in which processes are granted access to memory and for how long.
- ⑦ Processor Management - It decides order in which each process accesses the CPU and how much CPU time each process can use.
- ⑧ Device Management - Keeps track of all devices connected to the system. Decides which process can access which device for how long.
- ⑨ File Management

* CPU Register access takes one CPU cycle. Memory access takes ten while completing memory access CPU needs to stall.
 Remedy $\Rightarrow$ Add a fast memory b/w CPU & Main Memory $\Rightarrow$ Cache
 $\Rightarrow$ Memory buffer used to accom

Memory Management : speed differential $\Rightarrow$ Cache

Memory Organization and Memory Hierarchy



Memory is of 2 types

1) External Memory or Secondary memory -
 Comprising of Magnetic Disk, Optical disk, magnetic tapes i.e. peripheral storage devices. These are accessible by the processor via I/O Module.

2) Internal Memory or Primary Memory -
 Comprising of Main Memory, Cache Memory and CPU Register. These are directly accessible by the processor.

* As we move from top to bottom the capacity increases

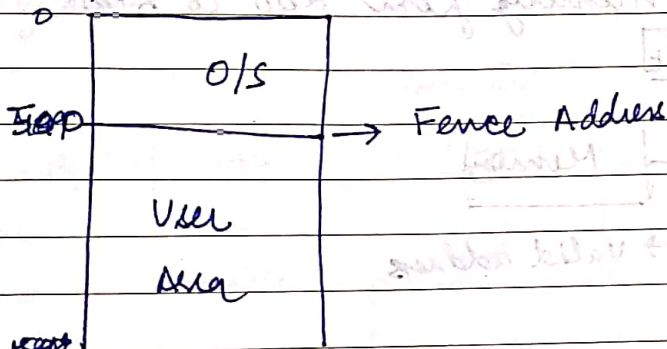
* Access Time or the interval b/w the read/write and request and availability of data increases from Top to Bottom

* Internal memory is more costly than External Memory.

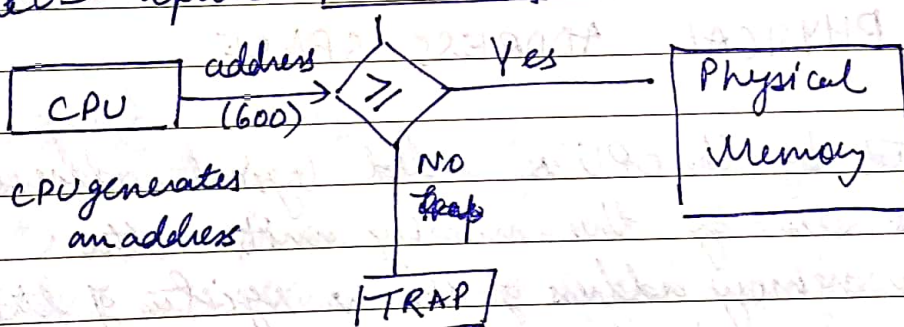
Memory Management - It is a functionality of the ^{Date} Operating system through which the OS manages primary memory and moves processes back and forth between main memory and disk during execution.

Functions of Memory Management -

- (i) Keeps track of every memory location
- (ii) Tracks whether memory is allocated or not
- (iii) Tracks how much memory is allocated
- (iv) Decides which process will get memory and when
- (v) Updates the status of memory location when it is allocated or free
- (vi) Ensures that O/S area is not accessible by users
A process cannot access OS or other processes memory



Fence Address - It is used to keep the user area and the OS apart. Fence Address

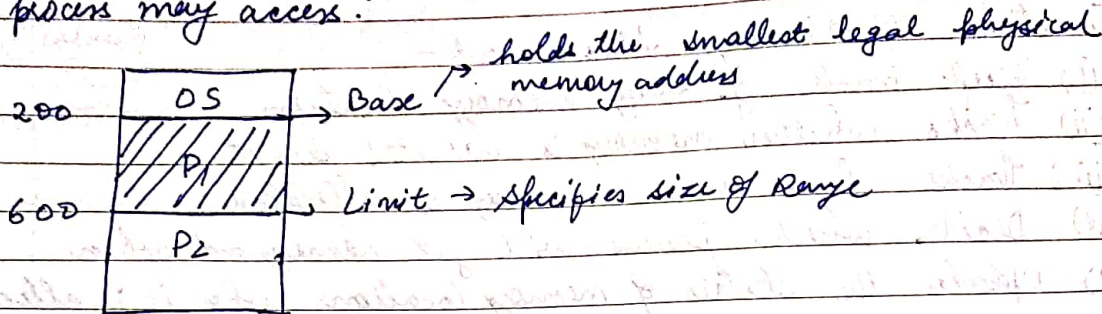


$600 > 500$ so it can be used by process.

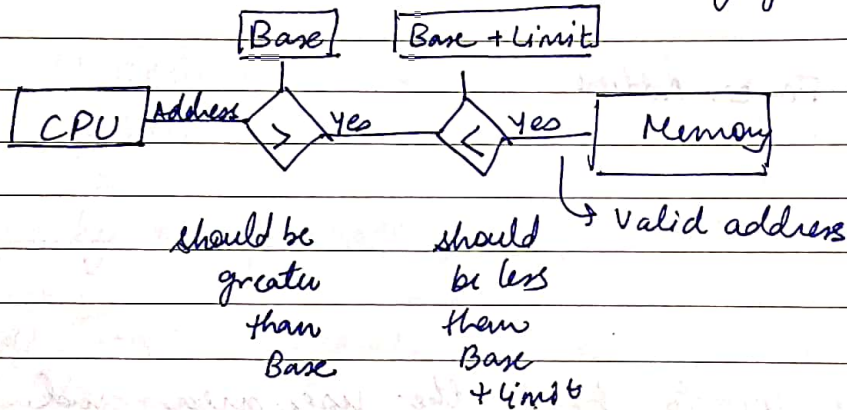
BASE and LIMIT REGISTERS

→ these are used so that each process has a separate memory space.

These are used to define range of legal addresses that a process may access.



Base = 200 Limit = 400
Process can access memory from 200 to 600



LOGICAL VS PHYSICAL ADDRESS SPACE

An address generated by the CPU is called logical address whereas the address seen by the memory unit - the one loaded into the memory address of the register of the memory is called physical address.

Logical Add → generated by CPU

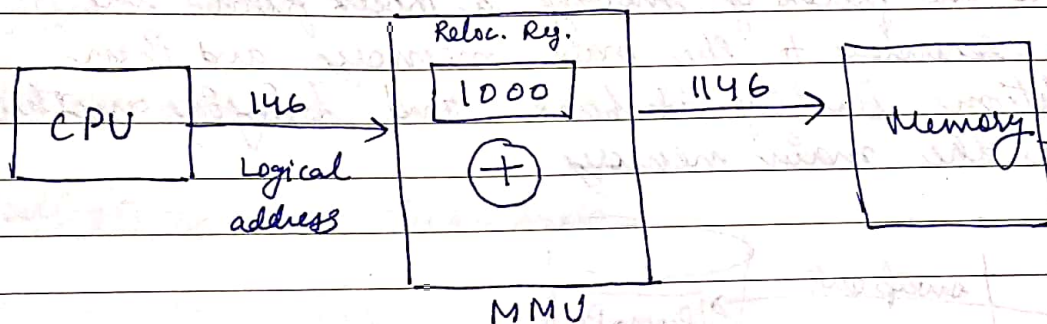
Phy. Address → Address seen by Memory unit

No. _____
Date _____
The compile time and load-time address binding generates identical physical & logical addresses.
But execution-time address binding results into different logical & physical addresses.

Set of all logical addresses $\rightarrow$ logical address space
Set of all physical addresses corresponding to these logical addresses $\rightarrow$ physical address space.

Run-time mapping from virtual to physical addresses is done by a hardware device called Memory Management Unit (MMU).

$\Rightarrow$ This run-time mapping can be done using relocation register.



DYNAMIC LINKING AND DYNAMIC LOADING

Dynamic Loading -

a routine is not loaded until it is called.

The main program is loaded into the memory & executed. When one routine needs to call another routine then that routine is loaded into the memory.

$\rightarrow$ advantage - unused routine is never loaded.
It enhances performance.

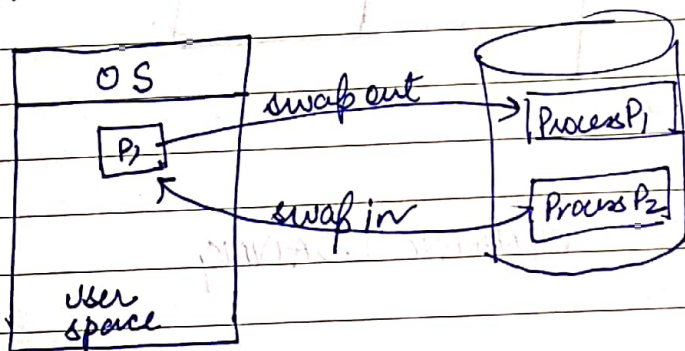
Dynamic Linking -

~~If it is dependent on other instead of loading both the li~~

It is similar to dynamic loading but here linking rather than loading is done at runtime. This feature is usually used with system libraries such as language subroutines libraries. Without this facility, each program on a system must include a copy of the language library. This wastes space.

SWAPPING -

Swapping is the process of moving a process from the main memory secondary to the main memory and then after completion sending it back and bringing another process in the main memory.



Rollout - Roll In - Priority based Scheduling

If a higher priority process arrives then a lower priority process is rolled out and memory is allocated to higher priority process which is rolled in.

There are 2 types of Memory Management Techniques

Date

Contiguous	Non Contiguous
→ It is centralised	It is decentralized
→ Program / process is stored in the memory as a single unit in contiguous memory locations	Program is stored in parts. Parts of the program are distributed in different locations on memory.
→ Implemented using partitioning	→ Implemented using paging, segmentation & Virtual Memory.

PARTITIONING:

Memory is divided into 2 partitions $\left\{ \begin{array}{l} \text{Resident OS} \\ \text{user processes} \end{array} \right.$

Partitioning is of two types

Fixed (MFT)
(Multiprogramming with fixed task)

variable (MVT)
(Multiprogramming with variable Task)

No of partitions are fixed $\Rightarrow$ diff. sizes

No of partitions are not fixed
partitions are made as per size of process.

1 Partition = 1 Process

MFT (Fixed Partitioning) Multiprogramming with Fixed Partitioning

No.
Date

Partition Allocation Policies

- (i) First Fit - allocates the process in the first free large enough partition
- (ii) Best Fit - Allocates the process in the smallest possible memory partition
- (iii) Worst Fit - Largest possible memory partition is allocated.
- (iv) Next Fit - It is same as first fit but it doesn't start from the first partition (starts where last partition allocated ends)

Example -

5 memory partitions - 100K, 500K, 200K, 300K, 600K
Processes P₁ 212K, P₂ 417K, P₃ 112K, P₄ 426K

First Fit

		100K
P ₄ doesn't get allocated	P ₁ 212K	500K
	P ₃	200K
		300K
	P ₂	600K

Worst Fit

		100K
P ₄ doesn't get allocated	P ₂	500K
		200K
	P ₃	300K
	P ₁	600K

Best Fit

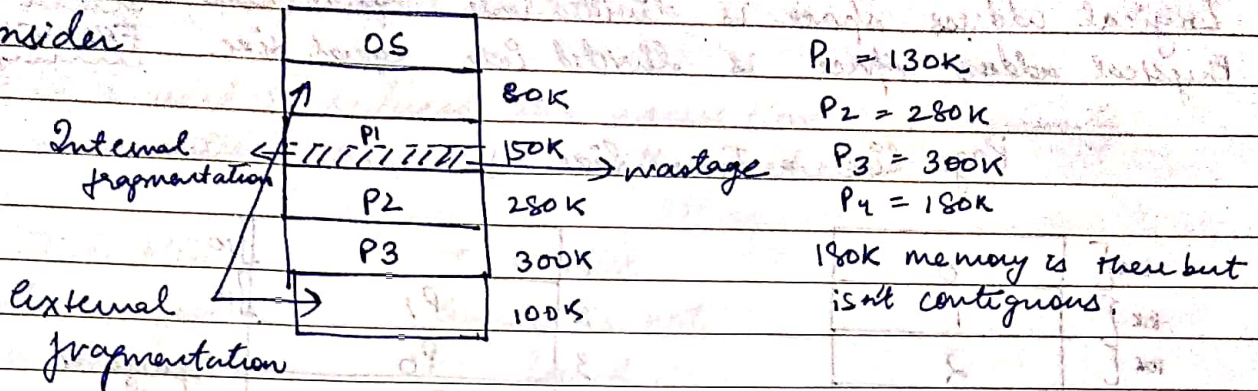
		100K
	P ₂	500K
	P ₃	200K
	P ₁	300K
	P ₄	600K

FRAGMENTATION: loss of memory

No.
Date

It is of two types → Internal
→ External

Consider



External fragmentation - Arises when unable to accommodate a process of a size even if we have sufficient size free but the memory bits are not contiguous.

Internal fragmentation - Arises when a process of size smaller than allocated partition size, ~~is placed~~ is placed in a partition, leaving behind some memory.

MVT (Multiprogramming with Variable Task) or Variable Partitioning

↳ Partitions are made according to the process size so there is no internal fragmentation.

External fragmentation can still be there.

NON CONTIGUOUS MEMORY ALLOCATION

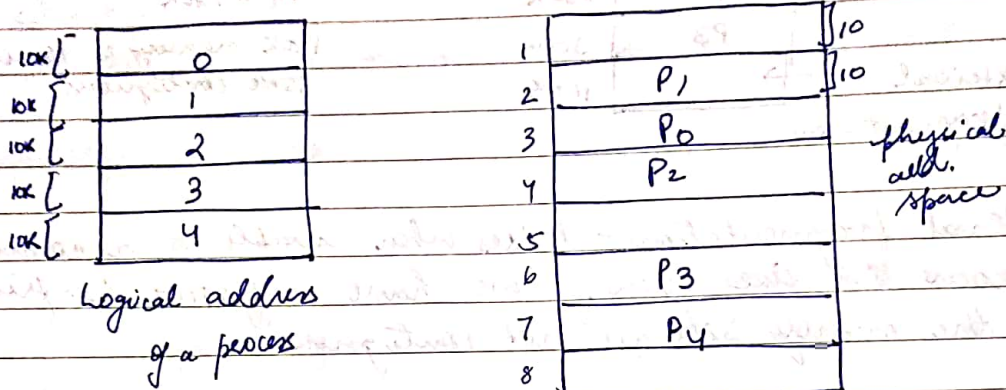
Done through Paging, Segmentation & Virtual Memory

① PAGING - No External Fragmentation

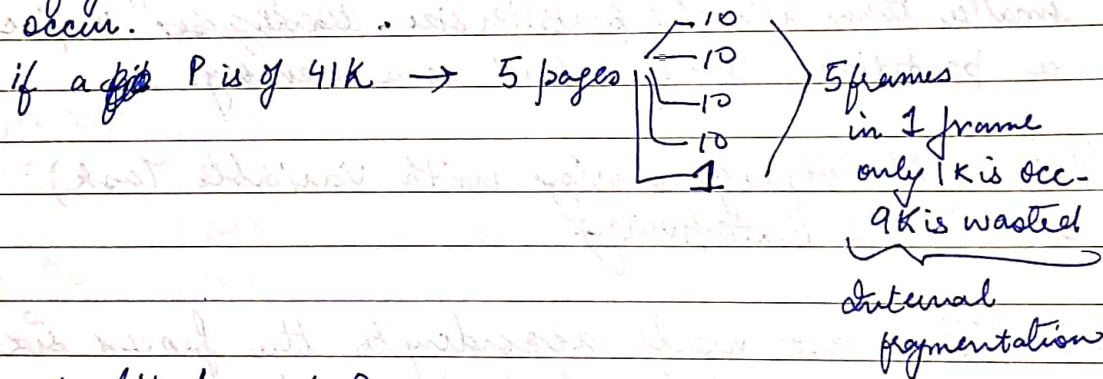
Logical address space is divided into equal size
Physical address space is divided into equal size

PAGES
FRAMES

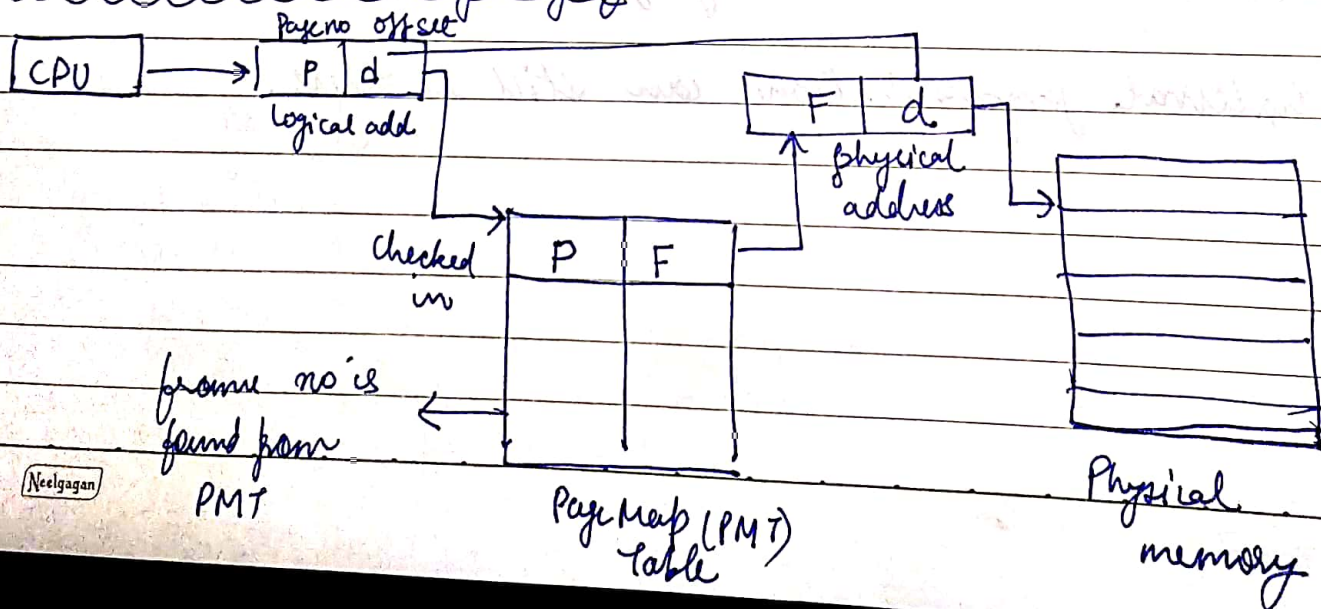
$$\text{Page Size} = \text{Frame Size}$$



External fragmentation is not there but Internal fragmentation can occur.



Hardware Architecture of Paging



Paging with Translation Look Aside Buffer (TLB)

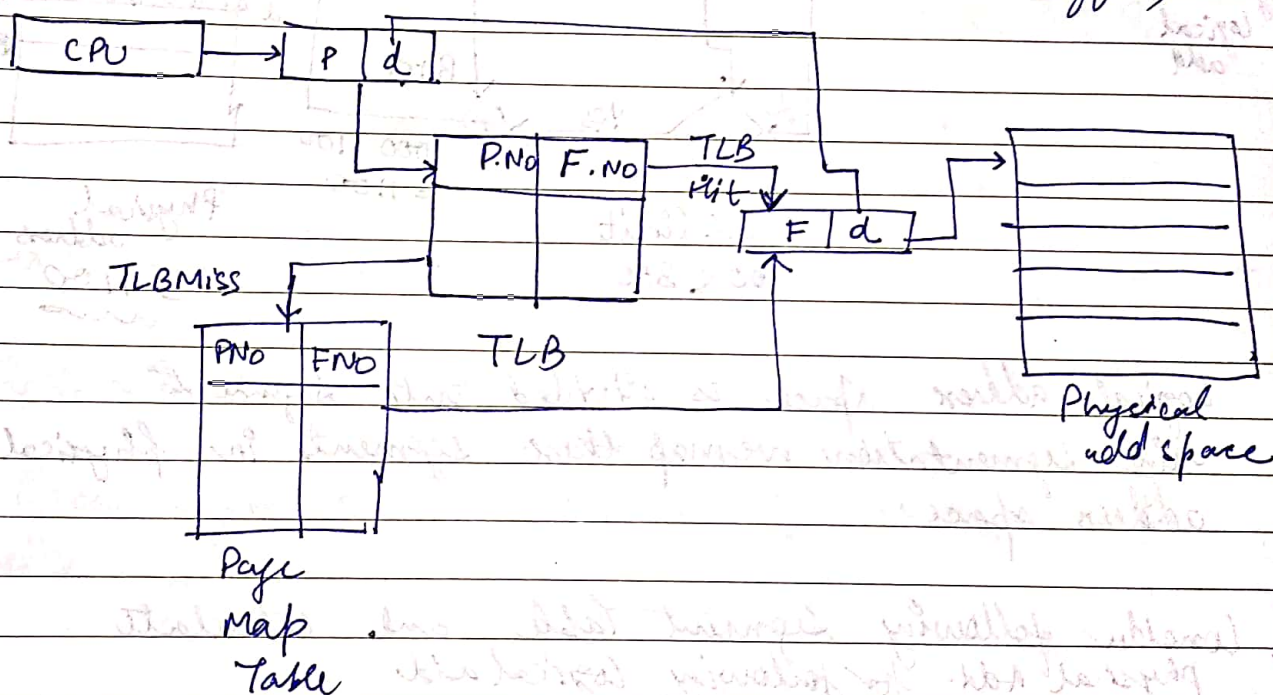
TLB $\rightarrow$ it is a hardware cache memory that contains very few page table entries.

$\rightarrow$ It is used to reduce the access time. Frequently used entries are kept in the cache.

TLB Hit
Entry is found in the cache memory

TLB Miss
Entry is not found

Architecture with TLB (Translation Look Aside Buffer)



Memory Protection in Paging: we associate **PROTECTION** bits and **Valid-Invalid** bits along with Page Table.

PROTECTION Bits (Read / Write, Read only)

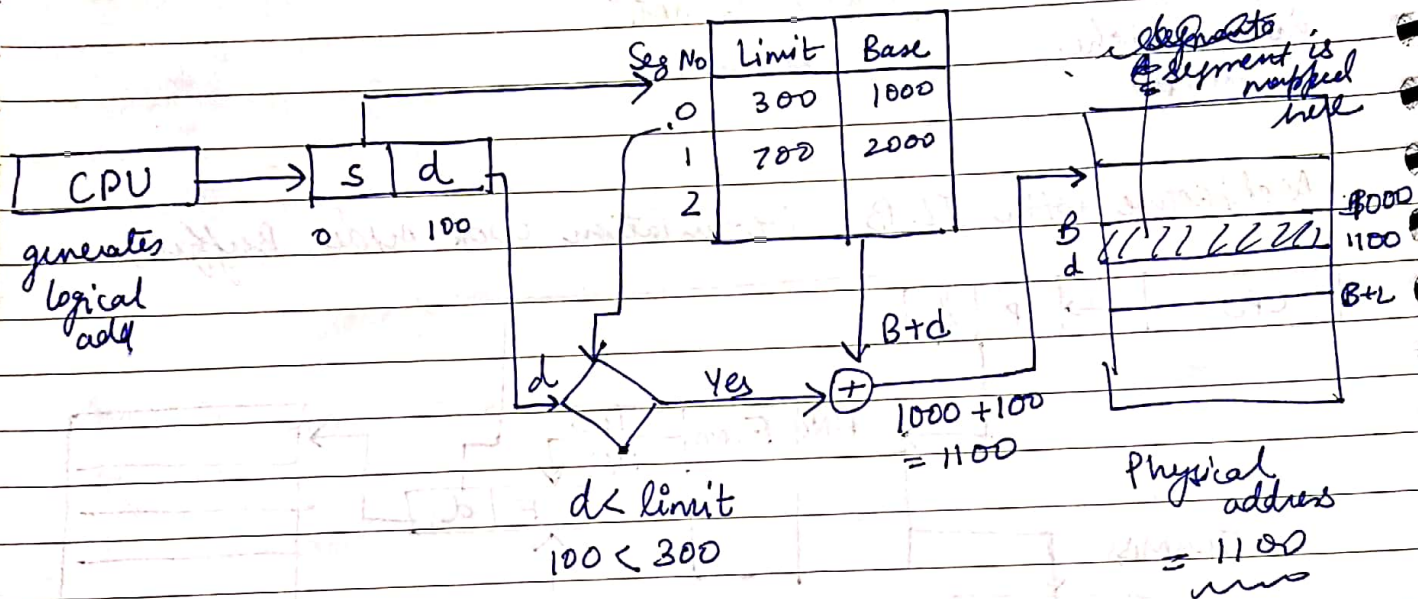
Valid-Invalid $\rightarrow$ **Valid** - Page is in Process logical address space
Invalid - Page is not in process logical address space

(3) SEGMENTATION - Non contiguous storage allocation technique

Segment Base $\Rightarrow$ Starting physical address
 Segment Limit $\Rightarrow$ size of segment

Each segment has a segment number and a length

Seg. No disp. $\rightarrow$ offset



Logical address space is divided into segments
 using segmentation we map these segments in physical address space

Q Consider following segment table and calculate physical Add. for following logical add

Segment	Base	Limit
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96

a) 0430

0 | 430

No.
Date

physical add = Base + disp

But $\text{disp} < \text{Limit}$

$$430 < 600$$

$$\text{physical add} = 219 + 430 = \underline{649}$$

b) 1 | 10

$$\text{physical add} = 141 + 10 = 24 \quad 2300 + 10 = \underline{2310}$$

c) 2 | 550

$$d = 550$$

$$d < \text{Limit}$$

$550 < 100$ False cannot map.

d) 3 | 400

$$d = 400$$

$$400 < 580 \quad \checkmark$$

$$\text{Physical add} = 1327 + 400 = \underline{1727}$$

— x — x — x — x — x — x — x —

SEGMENTATION with Pages

Each segment is divided into pages of fixed size

| S | P | d |

VIRTUAL MEMORY

No.

Date

It is a technique that allows execution of processes that are not completely in the memory.

Virtual memory abstracts main memory into a large uniform array of storage. This technique frees programmers from the concerns of memory storage limitations.

Advantages

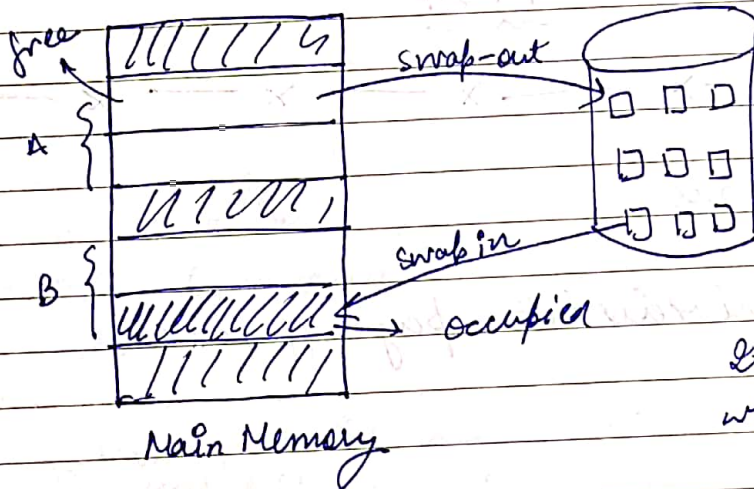
- only part of process needs to be in memory
- logical ^{add.} space is much larger than physical space

It is implemented using Demand Paging.

Page is brought into memory only when it is demanded.

Combination of swapping & Paging

demand of this approaching complete process should be in secondary storage



Lazy Swapper -
It swaps page only when needed.

- Advantages
- reduces memory req.
 - swap time reduced (as we swap page not the entire process)
 - inc. multiprogramming

Disadvantage - Page Fault [page req. but not present in main memory]

Performance Measurement

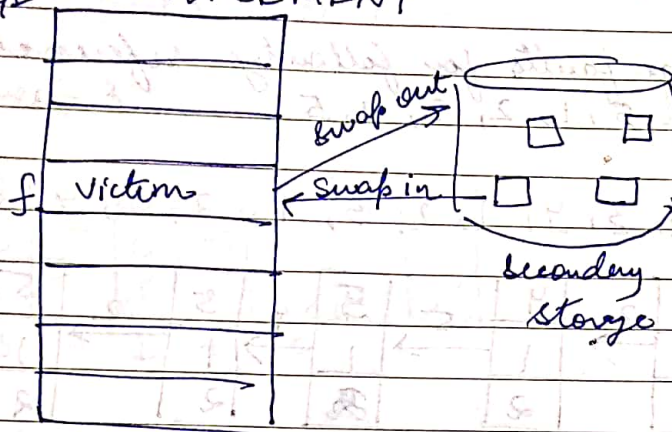
No.

Date

$$E_{\text{eff}} = \left[\overset{\substack{\text{prob of page fault} \\ p}}{p} \times \text{Page fault time} \right] + \left[(1-p) \times \underset{\substack{\text{general} \\ \text{memory access} \\ \text{time}}}{m_a} \right]$$

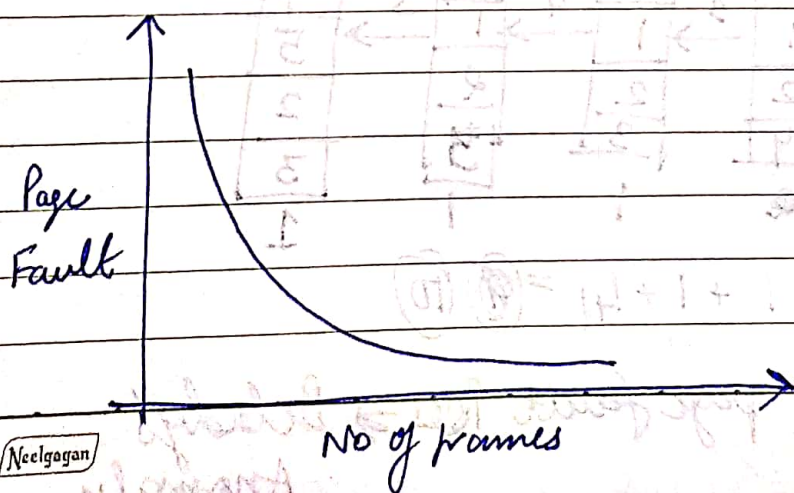
Effective mem. access time

PAGE REPLACEMENT



Page Ref Page Fault: It is a type of interrupt raised when a running process accesses a memory page that is mapped into virtual memory but not loaded in main memory.

Graph of Page Fault vs No of frames



Belady's Anomaly
when we increase
no. of frames, page
fault increases
whereas it should decrease.

PAGE REPLACEMENT ALGORITHMS

No.

Date

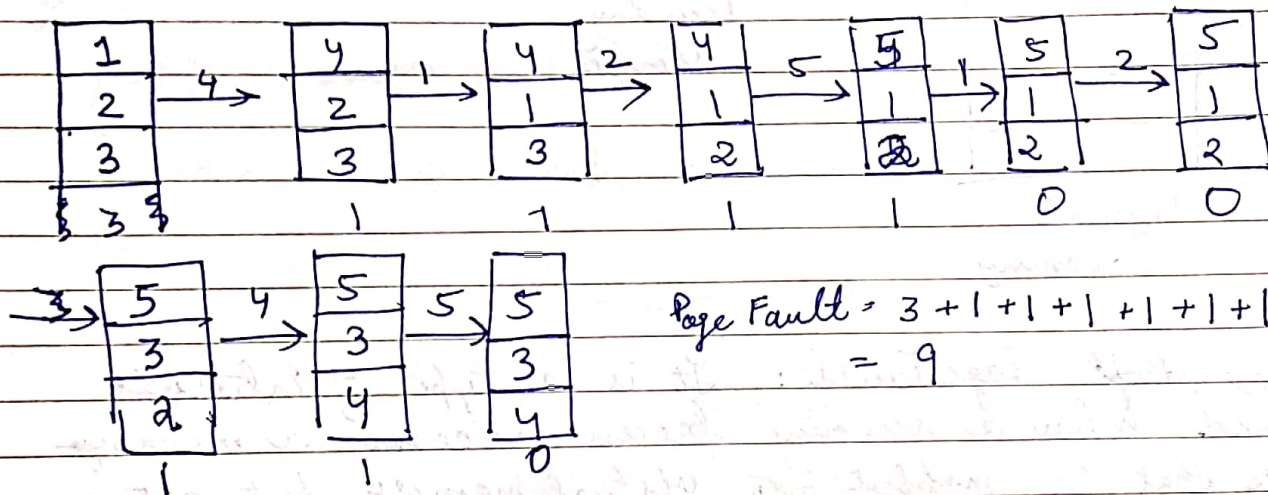
- ① First In First Out Algorithm
- ② Optimal Page Replacement Algorithm
- ③ Least Recently Used Algorithm

②

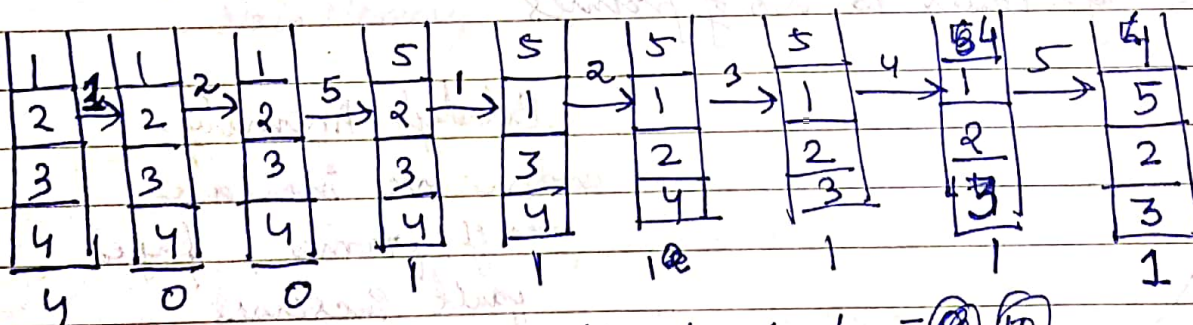
① FIRST IN FIRST OUT - the oldest page is replaced
→ performance is not good

Q Calculate no of page faults for following reference string 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5

(a) Using 3 Frames 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5



(b) Using 4 frames 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5



Page fault = 4 + 1 + 1 + 1 + 1 + 1 + 1 = 10

Even with ↑ in no of frames page fault ↑ ⇒ Belady's Anomaly.

② OPTIMAL PAGE REPLACEMENT ALGORITHM

↳ ① Lowest Page Fault

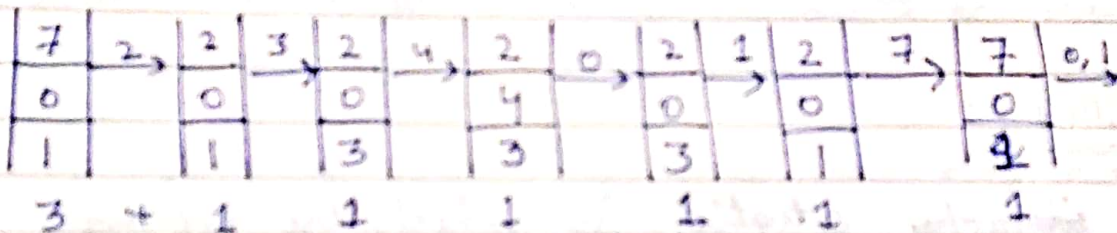
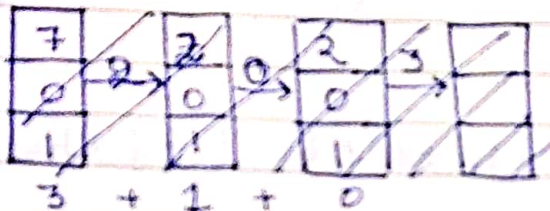
② Replace the page that will not be used for the longest amount of time.

It looks for pages in future. It is very difficult to implement.

Ex Calc. no. of page faults

7, 0, 1, 3, 0, 3, 0, 4, 3, 3, 0, 3, 2, 1, 3, 0
 ↳ 7, 0, 1

Frames = 3



page faults = 9

② OPTIMAL PAGE REPLACEMENT ALGORITHM

↳ ① Lowest Page Fault

② Replace the page that will not be used for the longest amount of time.

It looks for pages in future. It is very difficult to implement.

I Calc. no g page faults

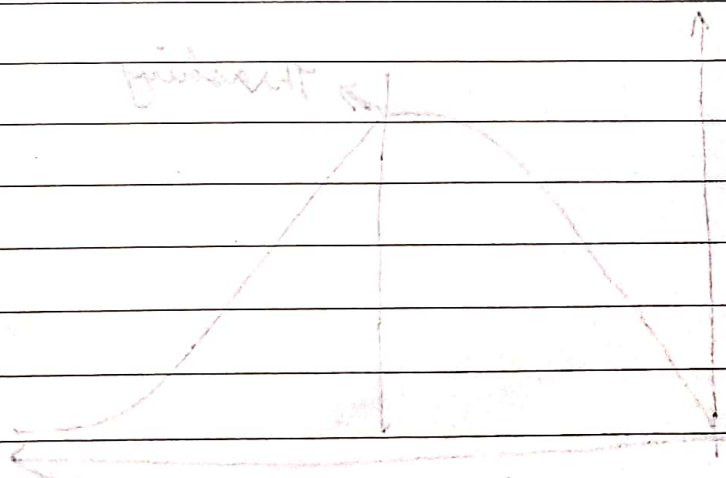
7, 0, 1, 3, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0
1, 7, 0, 1

1, 7, 0, 1

$$F_{\text{frames}} = 3$$

7	2	2	3	2	2	2	1	2	7	7
0		0		0	4	0		0		0
1		1		3	3	3		1		4
3	4	1		1	1	1		1		1

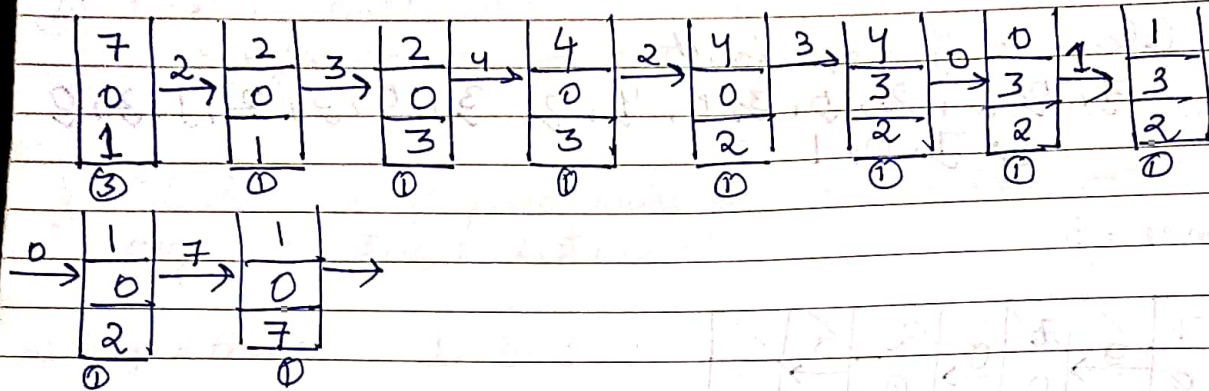
page fault = 9



③ LEAST RECENTLY USED ALGORITHM

Page that has not been used for the longest time in the main memory will be selected for replacement.

7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1



$$\text{page fault} = 3 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1 + 1$$

$$= 12$$

THRASHING -

It is a ~~situations~~ situation in which the system spends more time in paging instead of their execution. A process is busy swapping pages in and out. Instead of focussing on actual execution.



Causes of Thrashing -

① High degree of Multiprogramming -

If no. of processes keep on increasing then the frames allocated to each process will keep on decreasing. So less no. of frames are available to each process, so page fault increases. CPU will be wasting time in swapping in and swapping out pages.

② Lack of frames

Recovery from Thrashing

→ allow only a certain no. of processes to be brought into main memory.

→ If system is in thrashing state then suspend some processes.